

toggle_modes.js - Complete Documentation

File Location: src/js/toggle_modes.js

Purpose: Light/Dark mode theme switching

Dependencies: None

Used by: All pages that need theme toggling

Overview

The `toggle_modes.js` file implements a complete theme management system using a `ThemeManager` class. It handles switching between light and dark modes, persisting user preference in `localStorage`, and dynamically updating UI icons and styles.

ThemeManager Class

A class-based approach for managing application themes with persistent storage and automatic initialization.

Constructor

Purpose: Initialize theme manager and apply saved theme preference.

Code:

```
class ThemeManager {  
  constructor() {  
    this.theme = localStorage.getItem('theme') || 'light';  
    this.sunIcon = '☀';  
    this.moonIcon = '☾';  
    this.init();  
    this.updateThemeIcon();  
  }  
}
```

Initialization Flow:

Constructor called

↓

Check localStorage for saved theme

↓

If found: use saved theme

If not found: default to 'light'

↓

Set icon strings (sun/moon emojis)

↓

Call init() to setup

↓

Update button icon

Properties:

- `this.theme` - Current theme ('light' or 'dark')
- `this.sunIcon` - Emoji for light mode button (☀️)
- `this.moonIcon` - Emoji for dark mode button (🌙)

init()

Purpose: Initialize theme system by applying saved theme and setting up listeners.

Parameters: None

Returns: void

Code:

```
init() {  
    this.applyTheme(this.theme);  
    this.setupEventListeners();  
}
```

What it does:

1. Applies current theme to page
2. Attaches click handler to toggle button

setupEventListeners()

Purpose: Attach click handler to theme toggle button.

Parameters: None

Returns: void

Code:

```
setupEventListeners() {  
  const toggleBtnIndex = document.getElementById('themeToggleBtn');  
  
  if (toggleBtnIndex) {  
    toggleBtnIndex.addEventListener('click', () => this.toggle());  
  }  
}
```

What it does:

- Finds button with ID `themeToggleBtn`
- Attaches click listener to toggle theme
- Uses arrow function to maintain `this` context

HTML Required:

```
<button id="themeToggleBtn">  
  <i id="themeIcon"></i> Dark Mode  
</button>
```

updateThemeIcon()

Purpose: Update button icon to show current theme status.

Parameters: None

Returns: void

Code:

```
updateThemeIcon() {  
  const isDark = this.theme === 'dark';  
  const icon = document.getElementById('themeIcon');  
  if (icon) {  
    icon.className = isDark ? 'fa-solid fa-sun' : 'fa-solid fa-moon';  
  }  
}
```

Icon Logic:

```
If dark mode active:
    Show sun icon (☀) - indicates click to switch to light
If light mode active:
    Show moon icon (🌙) - indicates click to switch to dark
```

CSS Classes:

- fa-solid fa-sun - FontAwesome sun icon (light theme button)
- fa-solid fa-moon - FontAwesome moon icon (dark theme button)

toggle()

Purpose: Switch theme and save preference.

Parameters: None

Returns: void

Code:

```
toggle() {
    this.theme = this.theme === 'light' ? 'dark' : 'light';
    this.applyTheme(this.theme);
    localStorage.setItem('theme', this.theme);
}
```

Execution Flow:

```
toggle() called
  ↓
Switch theme value
  ↓
Apply theme to page
  ↓
Save to localStorage
  ↓
(Icon updated automatically via applyTheme)
```

Persistent Storage:

```
localStorage.setItem('theme', this.theme);
// Saved as: { 'theme': 'dark' }
// Persists across page reloads and browser sessions
```

toggleBtn()

Purpose: Toggle theme with explicit icon update (alternative method).

Parameters: None

Returns: void

Code:

```
toggleBtn() {  
  this.theme = this.theme === 'light' ? 'dark' : 'light';  
  this.applyTheme(this.theme);  
  this.updateThemeIcon();  
  localStorage.setItem('theme', this.theme);  
}
```

Difference from toggle():

- Explicitly calls `updateThemeIcon()`
- `toggle()` relies on icon update in `applyTheme()`
- Both achieve same result (extra safety)

applyTheme(themeName)

Purpose: Apply theme classes and update button UI.

Parameters:

- `themeName` (string) - 'light' or 'dark'

Returns: void

Code:

```
applyTheme(themeName) {  
  const body = document.body;  
  const btnIndex = document.getElementById('themeToggleBtn');  
  if (themeName === 'dark') {  
    body.classList.add('dark-mode');  
    if (btnIndex) {  
      btnIndex.textContent = this.sunIcon + ' Light Mode';  
      btnIndex.setAttribute('aria-label', 'Switch to light mode');  
    }  
  } else {  
    body.classList.remove('dark-mode');  
    if (btnIndex) {  
      btnIndex.textContent = this.moonIcon + ' Dark Mode';  
      btnIndex.setAttribute('aria-label', 'Switch to dark mode');  
    }  
  }  
}
```

What it does:

For Dark Mode:

1. Adds `dark-mode` class to body
2. Updates button text to sun icon + "Light Mode"
3. Sets ARIA label for accessibility

For Light Mode:

1. Removes `dark-mode` class from body
2. Updates button text to moon icon + "Dark Mode"
3. Sets ARIA label for accessibility

CSS Side:

```
/* Light mode (no dark-mode class) */
body {
  background: #ffffb1; /* Beige */
  color: #000;
}

/* Dark mode (with dark-mode class) */
body.dark-mode {
  background: #313131; /* Dark gray */
  color: #fff;
}
```

setTheme(themeName)

Purpose: Manually set theme to specific value.

Parameters:

- themeName (string) - 'light' or 'dark'

Returns: void

Code:

```
setTheme(themeName) {
  if (['light', 'dark'].includes(themeName)) {
    this.theme = themeName;
    this.applyTheme(this.theme);
    localStorage.setItem('theme', this.theme);
  }
}
```

Example Usage:

```
// Force light mode
themeManager.setTheme('light');

// Force dark mode
themeManager.setTheme('dark');

// Invalid - does nothing
themeManager.setTheme('auto'); // Not allowed
```

getTheme()

Purpose: Get current theme value.

Parameters: None

Returns: string ('light' or 'dark')

Code:

```
getTheme() {  
    return this.theme;  
}
```

Example Usage:

```
const currentTheme = themeManager.getTheme();  
  
if (currentTheme === 'dark') {  
    console.log('Dark mode active');  
}
```

Module Initialization

Conditional Initialization Pattern

```
let themeManager;  
  
if (document.readyState === 'loading') {  
    // Page is still loading  
    document.addEventListener('DOMContentLoaded', () => {  
        themeManager = new ThemeManager();  
    });  
} else {  
    // Page already loaded (script loaded late)  
    themeManager = new ThemeManager();  
}
```

Why Two Cases?

1. **Script loaded early:** `readyState === 'loading'`

- Need to wait for DOM to be ready
- Listen to `DOMContentLoaded` event

2. **Script loaded late:** `readyState === 'interactive' or 'complete'`

- DOM already ready
- Initialize immediately

This prevents:

- ☐ Can't find DOM elements (if waiting unnecessarily)
- ☐ Crashes on late-loaded scripts

Complete Usage Example

HTML Setup

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet" href="css/homepage.css">
</head>
<body>
  <nav>
    <button id="themeToggleBtn" class="theme-btn">
      <i id="themeIcon" class="fa-solid fa-moon"></i>
      Dark Mode
    </button>
  </nav>

  <div class="container">
    <h1>EventFinder</h1>
    <p>Discover events...</p>
  </div>

  <script src="js/toggle_modes.js"></script>
</body>
</html>
```

CSS Structure

```
/* Light mode variables (default) */
:root {
  --beige: #f8c89483;
  --light-bg: #ffffbf1;
  --dark-green: #184430;
}

/* Light mode styles */
body {
  background: linear-gradient(135deg, var(--beige) 0%, var(--light-beige) 75%);
  color: #000;
  transition: background 0.3s ease;
}

/* Dark mode styles */
body.dark-mode {
  --beige: #333;
  --light-bg: #222;
  --dark-green: #666;

  background: linear-gradient(135deg, #1a1a1a 0%, #2d2d2d 75%);
  color: #fff;
}

button {
  background: var(--dark-green);
  color: white;
}
```

JavaScript Usage

```
// Get current instance
const currentTheme = themeManager.getTheme();
console.log('Theme:', currentTheme);

// Programmatically change theme
function setDarkModeForUser() {
    themeManager.setTheme('dark');
}

// Force light mode on mobile
if (window.innerWidth < 768) {
    themeManager.setTheme('light');
}
```

Data Persistence

localStorage Structure

```
// When user switches to dark mode:
localStorage.setItem('theme', 'dark');

// Stored in browser as:
{
    'theme': 'dark'
}

// On next visit:
this.theme = localStorage.getItem('theme'); // Gets 'dark'
```

Browser Storage Details:

- **Storage Type:** Client-side browser storage
- **Size Limit:** ~5-10MB per domain
- **Persistence:** Survives browser restart
- **Cleared:** Manual cache clear or browser settings

localStorage vs sessionStorage

```
// Current: Uses localStorage (persists)
localStorage.setItem('theme', this.theme);

// Alternative: sessionStorage (session only)
sessionStorage.setItem('theme', this.theme);
// Would reset when browser closes
```

Integration with Page Styling

How CSS Responds to Dark Mode

Example from homepage.css:

```
/* Light mode (default, no dark-mode class) */
.navbar {
  background: rgba(255, 251, 241, 0.9);
  border-bottom: 1px solid rgba(24, 68, 48, 0.15);
}

.btn {
  background: var(--green-light);
  color: white;
}

/* Dark mode (with body.dark-mode class) */
body.dark-mode .navbar {
  background: rgba(49, 49, 49, 0.95);
  border-bottom: 1px solid rgba(255, 255, 255, 0.15);
}

body.dark-mode .btn {
  background: #555;
  color: #fff;
}
```

Automatic Application:

1. User clicks theme toggle
2. `toggle()` adds/removes `dark-mode` class
3. CSS cascade automatically applies matching styles
4. Page recolors instantly

Advanced Features

Respecting User Preferences

Enhancement for respecting OS dark mode preference:

```
// Check if user prefers dark mode in OS
const prefersDark = window.matchMedia('(prefers-color-scheme: dark)').matches;

// Use as default
constructor() {
  this.theme = localStorage.getItem('theme') ||
    (prefersDark ? 'dark' : 'light');
}

// Listen for OS preference changes
window.matchMedia('(prefers-color-scheme: dark)').addEventListener('change', (e) => {
  if (!localStorage.getItem('theme')) { // Only if user hasn't set preference
    this.applyTheme(e.matches ? 'dark' : 'light');
  }
});
```

Scheduled Theme Switching

Change theme based on time of day:

```
function applyScheduledTheme() {
  const hour = new Date().getHours();

  // Dark mode from 6 PM to 6 AM
  if (hour >= 18 || hour < 6) {
    themeManager.setTheme('dark');
  } else {
    themeManager.setTheme('light');
  }
}

// Apply on load
applyScheduledTheme();

// Check every hour
setInterval(applyScheduledTheme, 60 * 60 * 1000);
```

Accessibility Considerations

ARIA Labels

Code includes proper accessibility attributes:

```
btnIndex.setAttribute('aria-label', 'Switch to light mode');
// Screen readers announce: "Button: Switch to light mode"
```

Keyboard Navigation

```
// Button is keyboard accessible by default
// Users can Tab to button and press Enter/Space
document.getElementById('themeToggleBtn').addEventListener('click', () => {
  themeManager.toggle();
});

// Also works with keyboard
```

Color Contrast

Dark mode should maintain WCAG AA contrast:

```
/* Good contrast in dark mode */
body.dark-mode {
  background: #1a1a1a; /* Very dark */
  color: #f0f0f0;      /* Very light */
  /* Ratio: 15:1 ✓ */
}
```

Browser Support

- ☐ Chrome/Edge 50+ (localStorage, classList)
- ☐ Firefox 45+
- ☐ Safari 10+
- ☐ IE 11 (with polyfills)

Required APIs:

- `localStorage` - Browser storage
 - `classList` - DOM class manipulation
 - `querySelector()` - Element selection
-

Performance

Optimizations

- ☐ No external dependencies
- ☐ Single class instance (no memory waste)
- ☐ CSS handles styling (fast, GPU-accelerated)
- ☐ `localStorage` access minimal
- ☐ No DOM manipulation on every frame

Bundle Size

- **Original:** ~1.2 KB
 - **Minified:** ~0.6 KB
 - **Impact:** Negligible
-

Summary

`toggle_modes.js` provides:

Feature	Benefit
Class-based Design	Organized, reusable code
Persistent Storage	User preference saved
Instant Switching	No page reload needed
Icon Updates	Visual feedback to user
Accessibility	Screen reader support
CSS Integration	Fast, efficient styling

The theme system creates a pleasant user experience with persistent preferences and instant visual feedback.

Last Updated: May 22, 2026

File Size: ~100 lines of code

Complexity: Low-Medium

Criticality: Medium (UX enhancement)